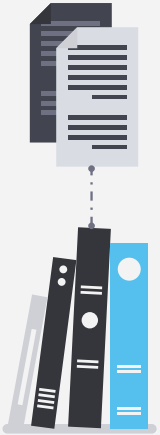
Secure LLM Assistant

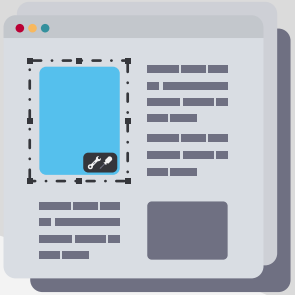
COMP.SEC.300

Ville Viinikka

Secure LLM Assistant

- Secure LLM application running on CSC cloud
- React and Nginx frontend with FastAPI backend and Ollama runtime
- Core focus on JWT sessions 2FA and automated DevSecOps pipelines



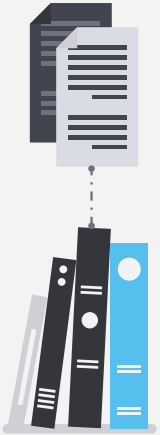


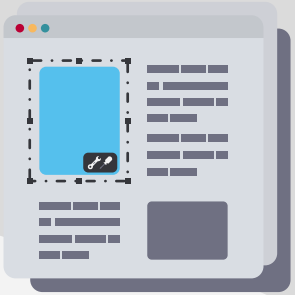
02

Project Goal

Project Goal

- Build a local LLM assistant safely
- Establish a strict backend security layer in front of the model
- Prevent direct model exposure to the user
- Combine secure coding privacy authentication and DevSecOps



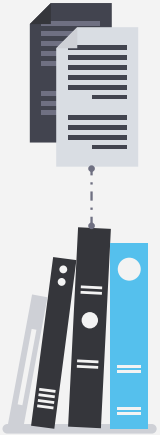


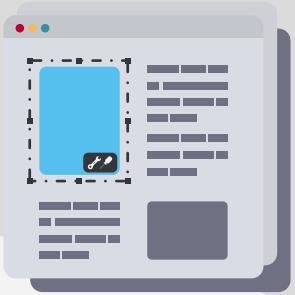
03

Earlier Work

Earlier Work

- Based on a standard local LLM chat architecture but the project is a new one.
- Extended with security features and implementation

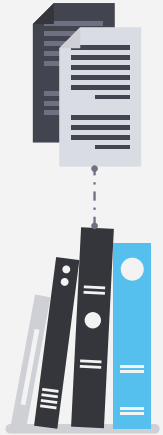




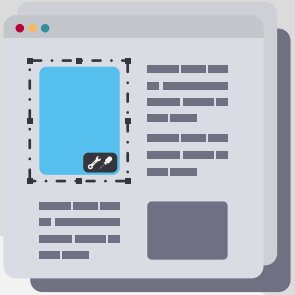
04

Technologies Used

Technologies Used



- Backend: Python, FastAPI, Pydantic, httpx, SQLite, PyJWT, pyotp
- Frontend: JavaScript, React, Vite, Nginx
- LLM runtime: Ollama
- DevOps: Docker Compose, Jenkins, Dependabot
- Testing and security tools: pytest, Bandit, pip-audit, npm audit

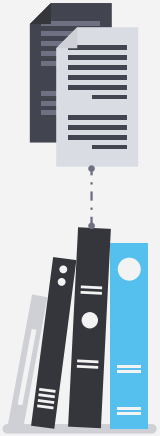


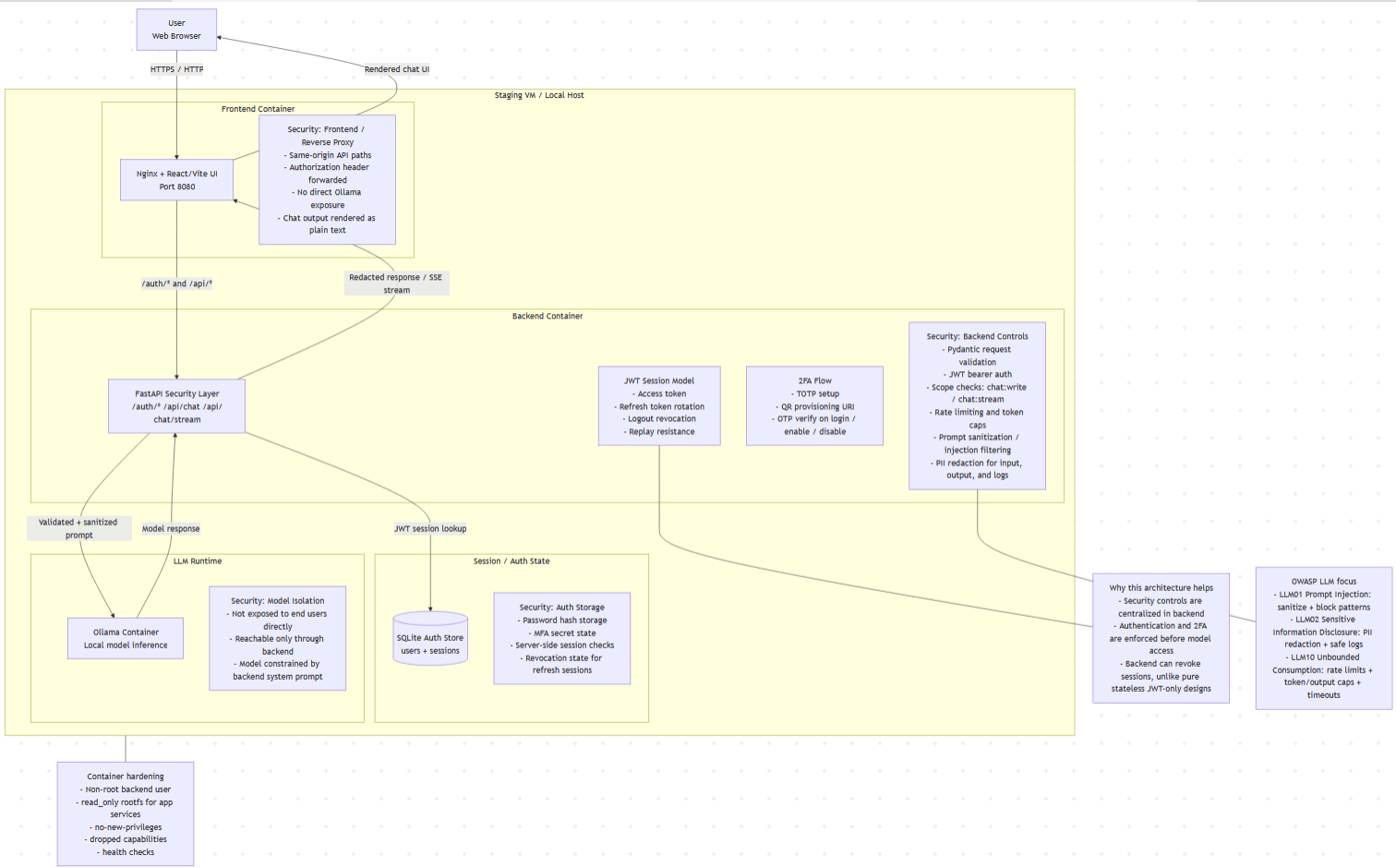
05

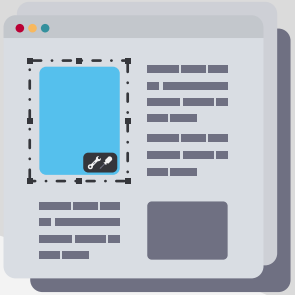
Architecture

Architecture

- User connects to Frontend which routes to Backend which queries Ollama
- Backend serves as the absolute trust boundary
- Validation authentication redaction and limits execute before model access




User
Web Browser
HTTPS / HTTP
Rendered chat UI
Staging VM / Local Host
Frontend Container
Security: Frontend / Reverse Proxy
- Same-origin API paths
- Authorization header forwarded
- No direct Ollama exposure
- Chat output rendered as plain text
Nginx + React/Vite UI
Port 8080
/auth/* and /api/*
Redacted response / SSE stream
Backend Container
FastAPI Security Layer
/auth/* /api/chat /api/chat/stream
JWT Session Model
- Access token
- Refresh token rotation
- Logout revocation
- Replay resistance
2FA Flow
- TOTP setup
- QR provisioning URI
- OTP verify on login / enable / disable
Security: Backend Controls
- Pydantic request validation
- JWT bearer auth
- Scope checks: chat:write / chat:stream
- Rate limiting and token caps
- Prompt sanitization / injection filtering
- PII redaction for input, output, and logs
Validated + sanitized prompt
Model response
JWT session lookup
LLM Runtime
Ollama Container
Local model inference
Security: Model Isolation
- Not exposed to end users directly
- Reachable only through backend
- Model constrained by backend system prompt
Session / Auth State
SQLite Auth Store
users + sessions
Security: Auth Storage
- Password hash storage
- MFA secret state
- Server-side session checks
- Revocation state for refresh sessions
Why this architecture helps
- Security controls are centralized in backend
- Authentication and 2FA are enforced before model access
- Backend can revoke sessions, unlike pure stateless JWT-only designs
OWASP LLM focus
- LLM01 Prompt Injection: sanitize + block patterns
- LLM02 Sensitive Information Disclosure: PII redaction + safe logs
- LLM10 Unbounded Consumption: rate limits + token/output caps + timeouts
Container hardening
- Non-root backend user
- read_only rootfs for app services
- no-new-privileges
- dropped capabilities
- health checks

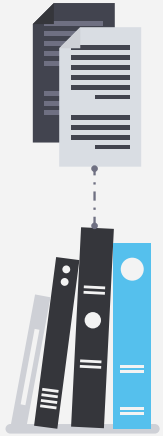


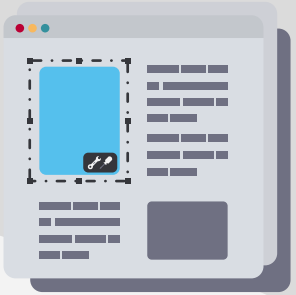
06

Security Controls

Security Controls

- Prompt sanitization and prompt injection filtering
- PII redaction for prompts responses and application logs
- Request validation using Pydantic models
- Rate limiting token caps and strict timeout handling



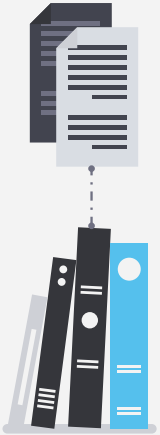


07

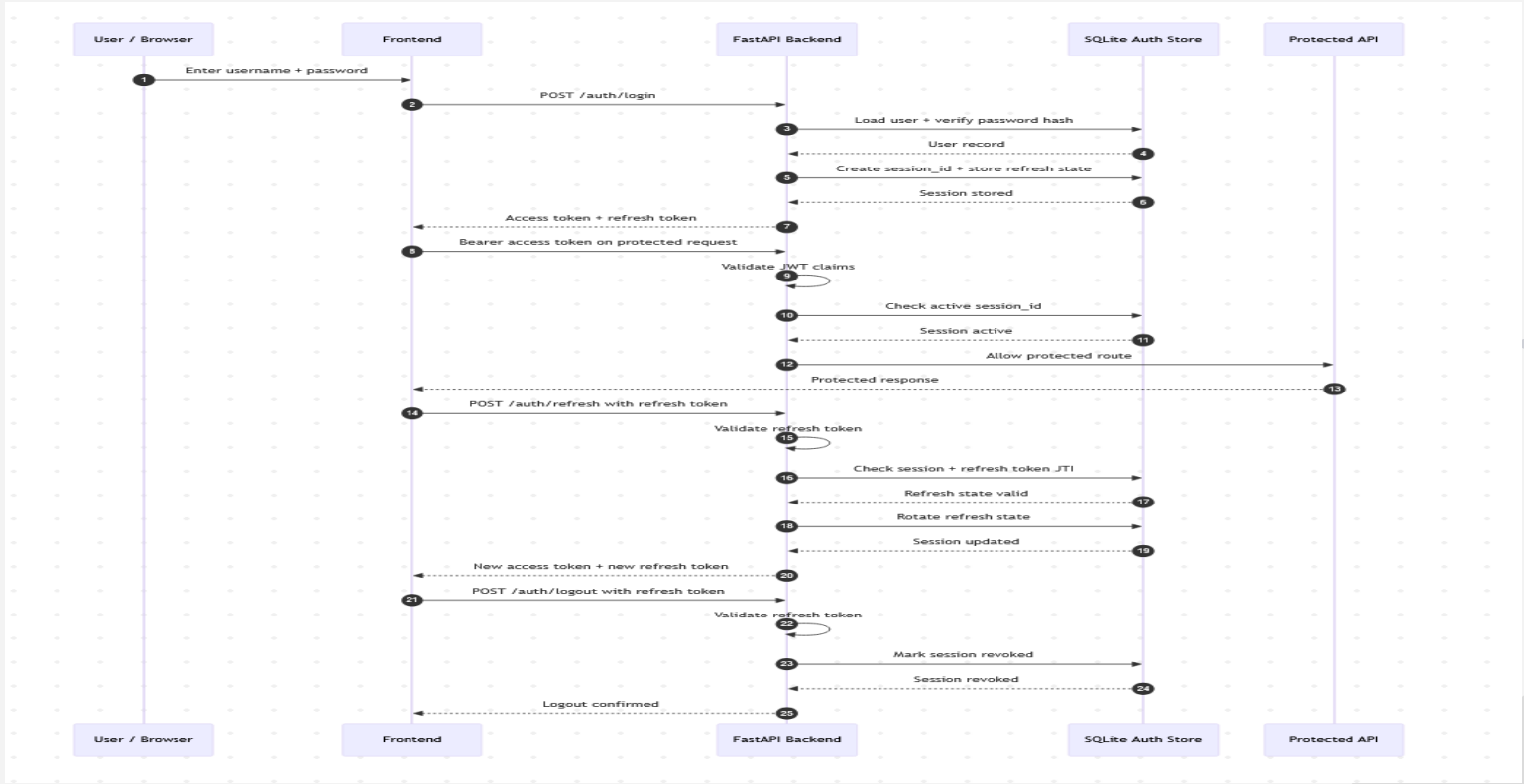
JWT Session Database

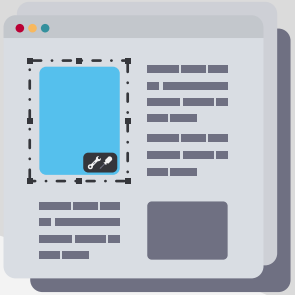
JWT Session Database

- Access token and refresh token model
- SQLite backed server side session store
- Refresh token rotation
- Logout revocation and replay resistance
- Login salted, hashed 240,000 iterations



JWT Session Database



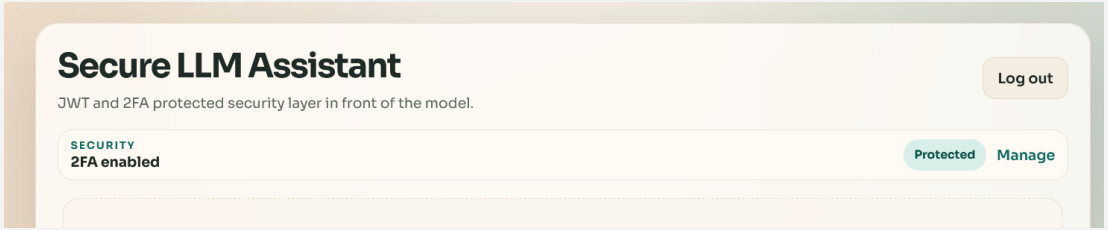


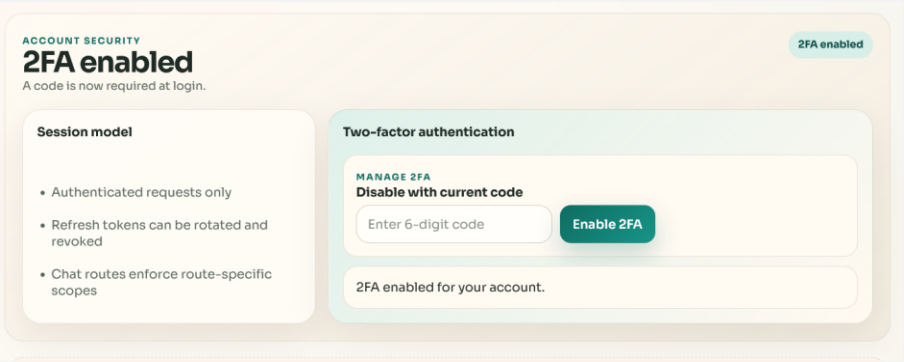
08

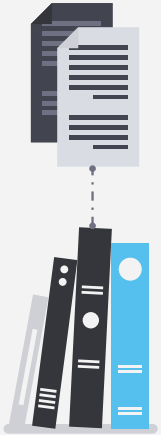
2FA Implementation

2FA Implementation

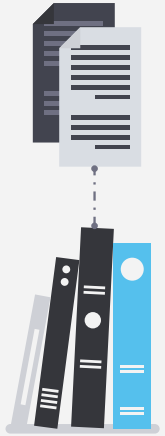
- TOTP implementation using pyotp
- OTP required on login when enable







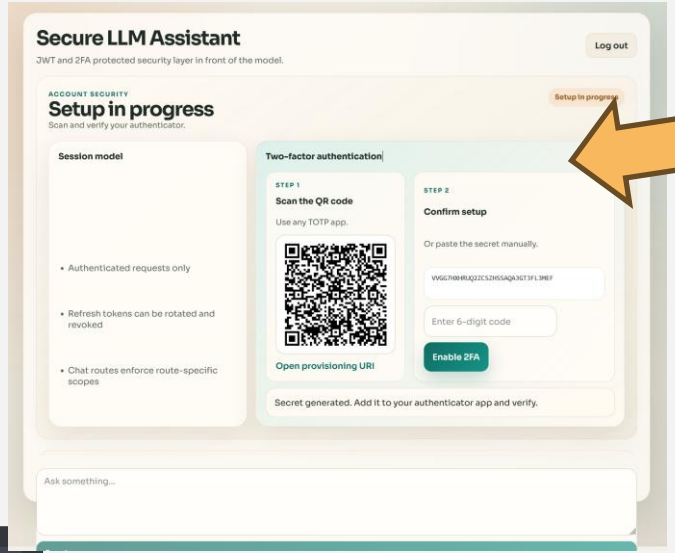
2FA Implementation

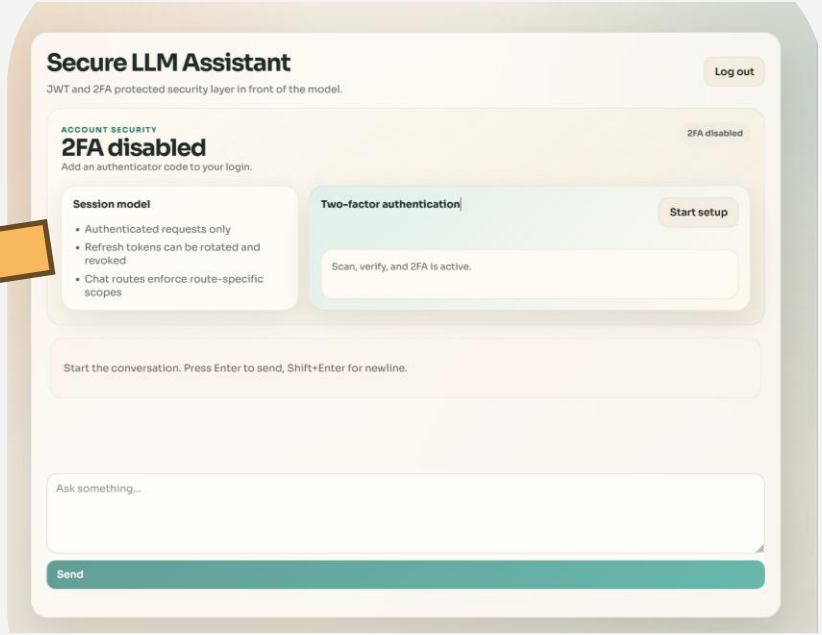


- TOTP implementation using pyotp
- OTP required on login when enable

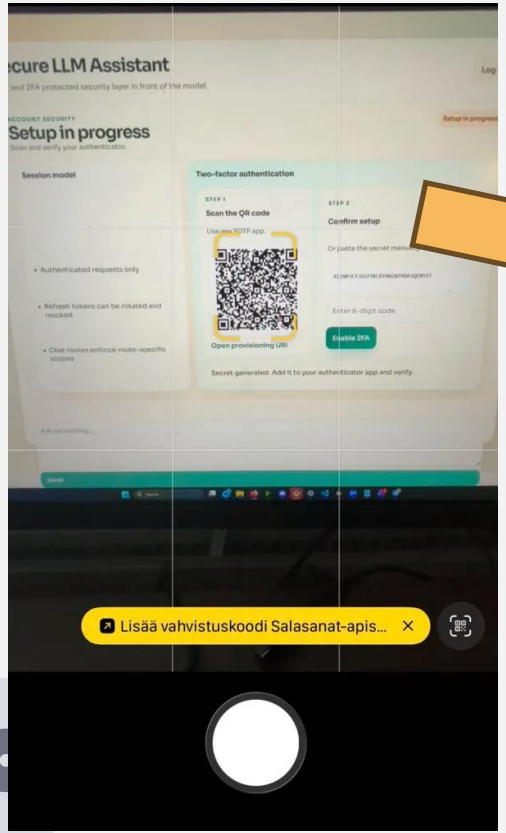
1. Create user
2. Start 2FA setup
3. Scan the QR code
4. Setup the 2FA on mobile phone
5. 2FA setup complete

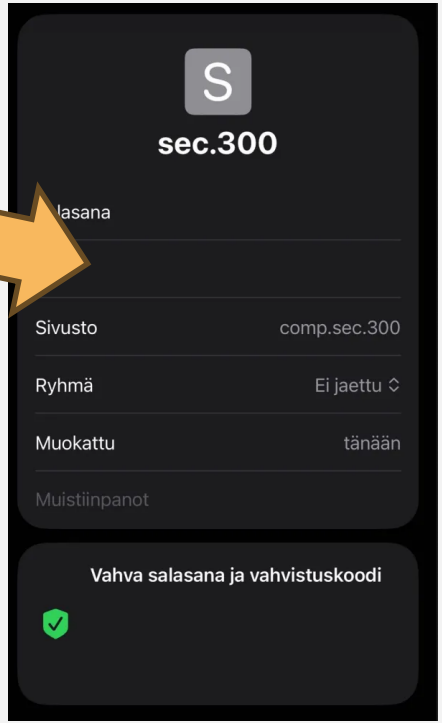
2FA Demo

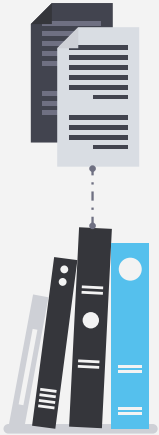

Secure LLM Assistant
JWT and 2FA protected security layer in front of the model.
Log out
ACCOUNT SECURITY
Setup in progress
Scan and verify your authenticator.
Setup in progress
Session model
Authenticated requests only
Refresh tokens can be rotated and revoked
Chat routes enforce route-specific scopes
Two-factor authentication
STEP 1
Scan the QR code
Use any TOTP app.
Open provisioning URI
STEP 2
Confirm setup
Or paste the secret manually.
Enter 6-digit code
Enable 2FA
Secret generated. Add it to your authenticator app and verify.
Ask something...


Secure LLM Assistant
JWT and 2FA protected security layer in front of the model.
Log out
ACCOUNT SECURITY
2FA disabled
Add an authenticator code to your login.
2FA disabled
Session model
Authenticated requests only
Refresh tokens can be rotated and revoked
Chat routes enforce route-specific scopes
Two-factor authentication
Start setup
Scan, verify, and 2FA is active.
Start the conversation. Press Enter to send, Shift+Enter for newline.
Ask something...
Send

2FA Demo

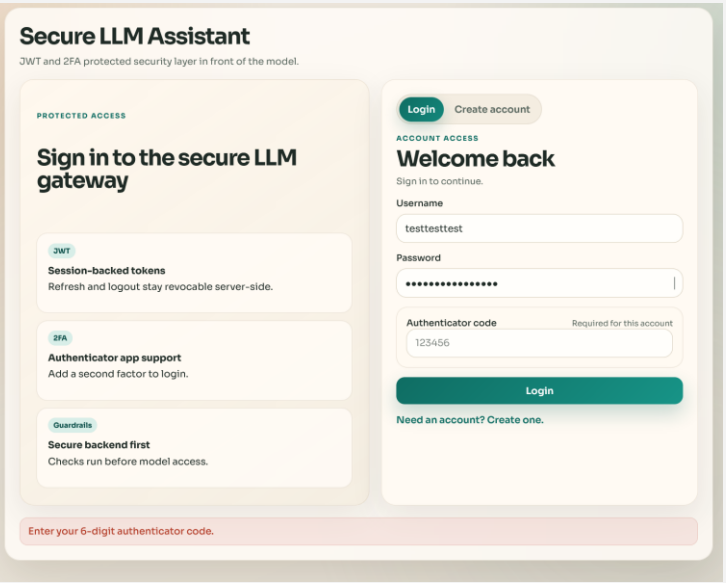

cure LLM Assistant
Setup in progress
Session model
Two-factor authentication
Scan the QR code
Confirm setup
Enable 2FA
Lisää vahvistuskoodi Salasanat-apis...

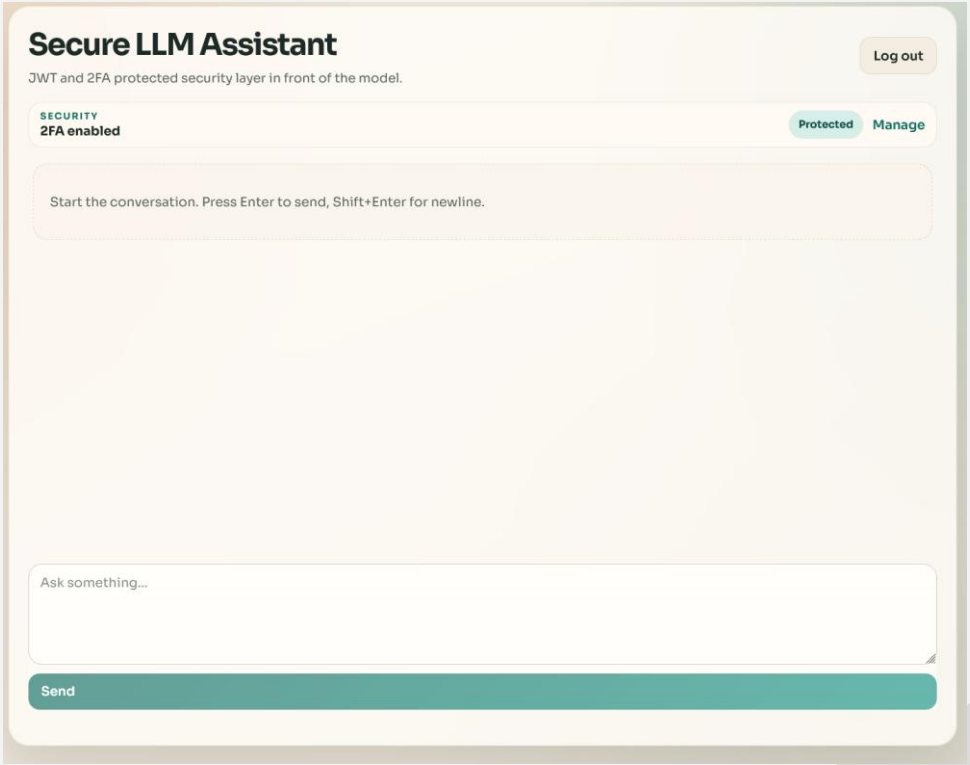

S
sec.300
Sivusto
comp.sec.300
Ryhmä
Ei jaettu
Muokattu
tänään
Muistiinpanot
Vahva salasana ja vahvistuskoodi

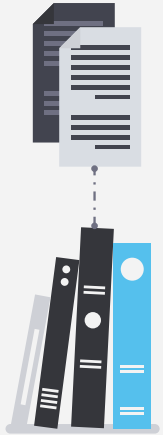


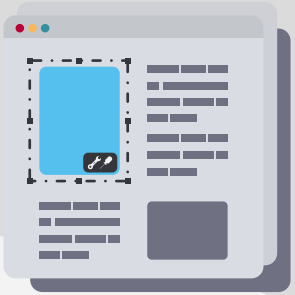
2FA Demo

Completed
Next time login asks for 2FA





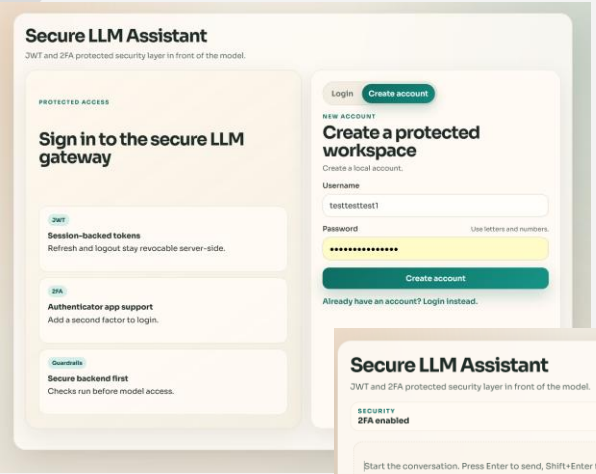


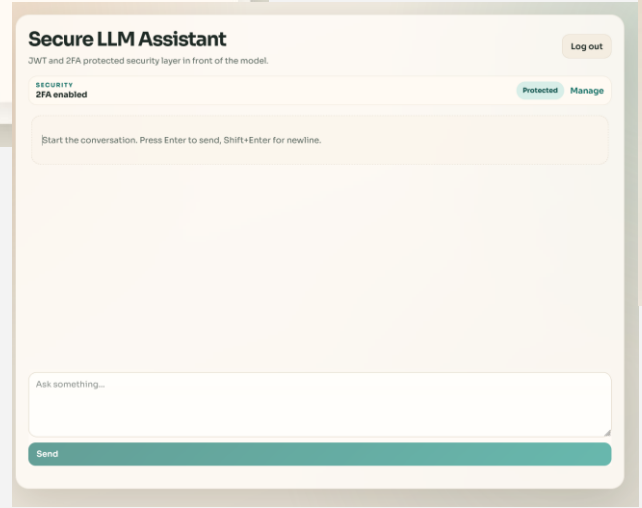


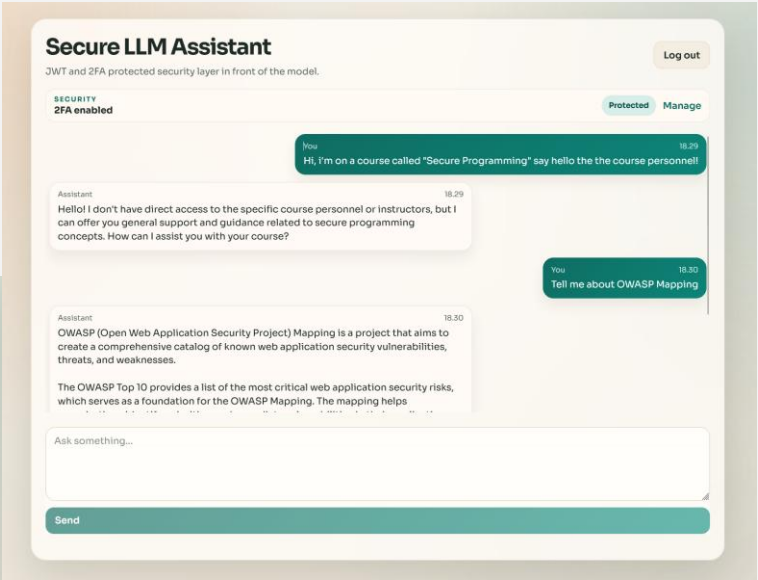
09

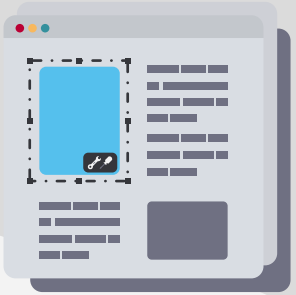
Application Demo

Application Demo







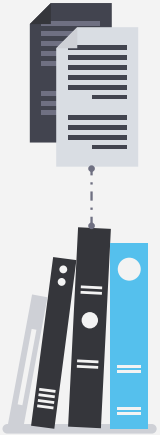


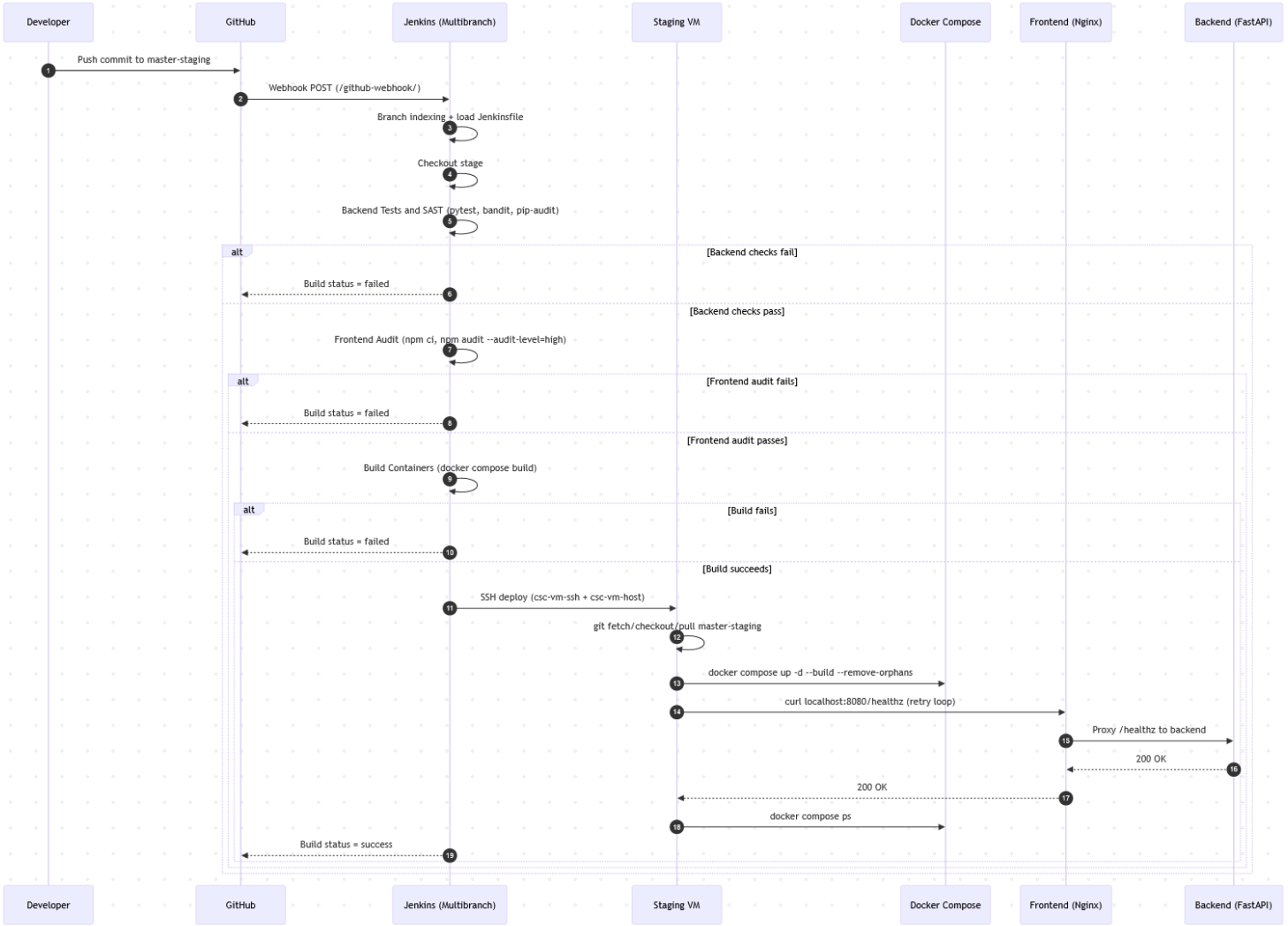
10

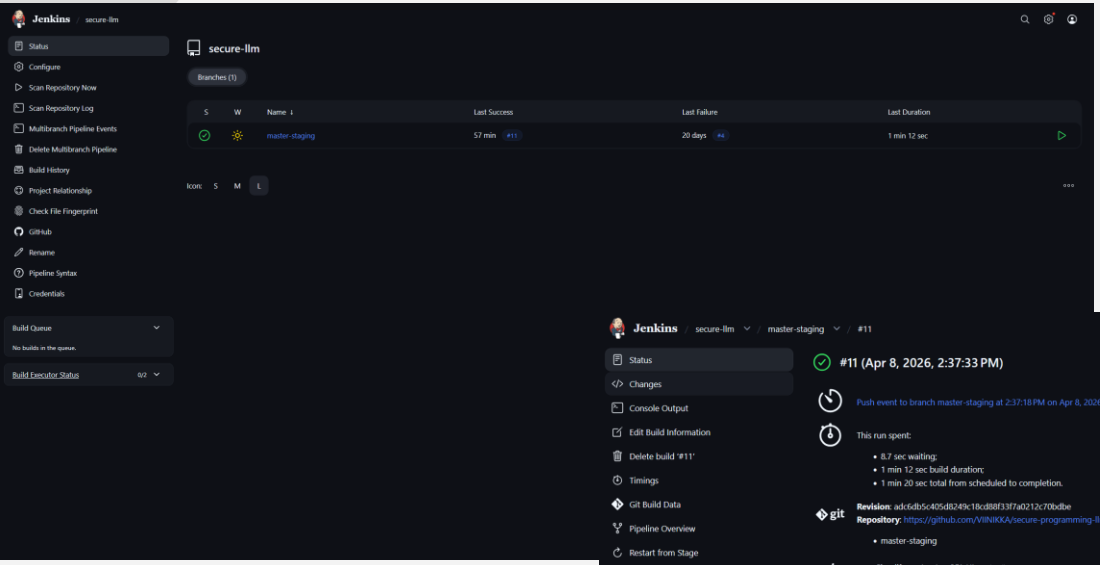
CI CD Pipeline

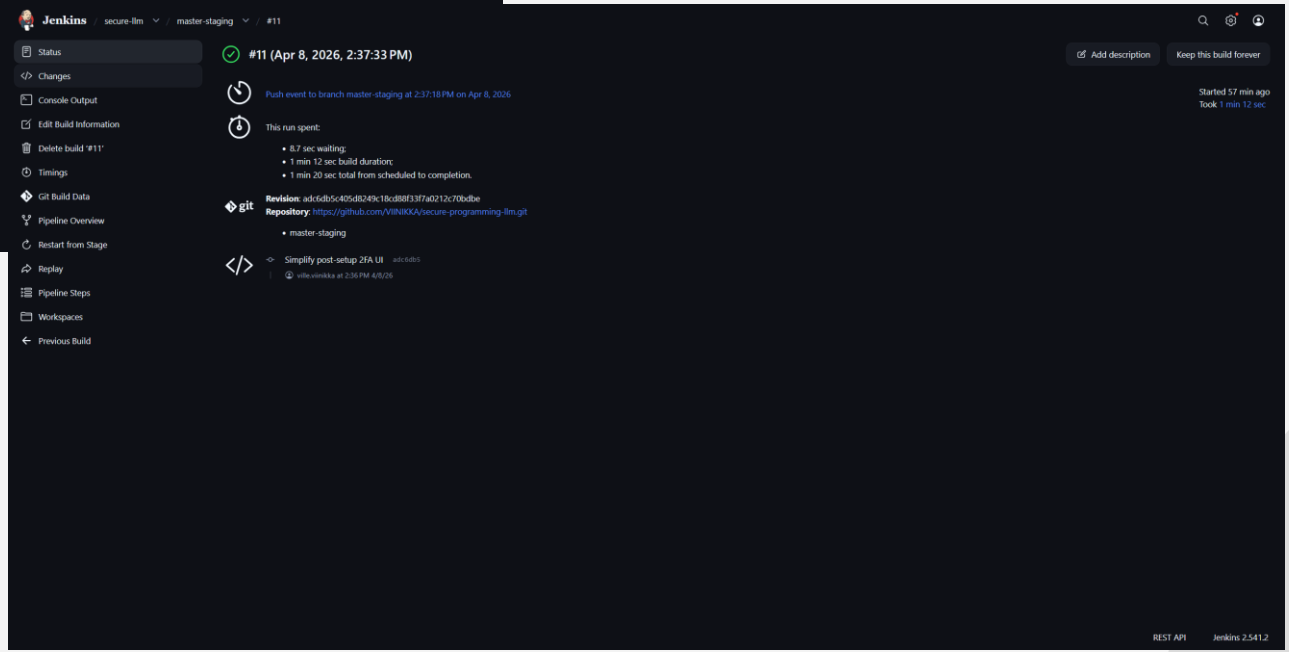
CI CD Pipeline

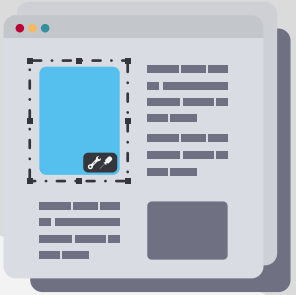
- Jenkins automated pipeline runs tests and security checks
- Executes pytest bandit pip audit and npm audit
- Automates container build and deployment to a staging environment
- Verifies deployment with an automated health check




Developer
GitHub
Jenkins (Multibranch)
Staging VM
Docker Compose
Frontend (Nginx)
Backend (FastAPI)
1
Push commit to master-staging
2
Webhook POST (/github-webhook/)
3
Branch indexing + load Jenkinsfile
4
Checkout stage
5
Backend Tests and SAST (pytest, bandit, pip-audit)
alt
[Backend checks fail]
Build status = failed
6
[Backend checks pass]
7
Frontend Audit (npm ci, npm audit --audit-level=high)
alt
[Frontend audit fails]
Build status = failed
8
[Frontend audit passes]
9
Build Containers (docker compose build)
alt
[Build fails]
Build status = failed
10
[Build succeeds]
11
SSH deploy (csc-vm-ssh + csc-vm-host)
12
git fetch/checkout/pull master-staging
13
docker compose up -d --build --remove-orphans
14
curl localhost:8080/healthz (retry loop)
15
Proxy /healthz to backend
16
200 OK
17
200 OK
18
docker compose ps
19
Build status = success
Developer
GitHub
Jenkins (Multibranch)
Staging VM
Docker Compose
Frontend (Nginx)
Backend (FastAPI)


Jenkins
secure-llm
Status
Configure
Scan Repository Now
Scan Repository Log
Multibranch Pipeline Events
Delete Multibranch Pipeline
Build History
Project Relationship
Check File Fingerprint
GitHub
Rename
Pipeline Syntax
Credentials
Build Queue
No builds in the queue.
Build Executor Status
0/2
secure-llm
Branches (1)
S
W
Name ↓
Last Success
Last Failure
Last Duration
master-staging
57 min #11
20 days #4
1 min 12 sec
Icon: S M L


Jenkins
secure-llm
master-staging
#11
Status
Changes
Console Output
Edit Build Information
Delete build '#11'
Timings
Git Build Data
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
#11 (Apr 8, 2026, 2:37:33 PM)
Add description
Keep this build forever
Push event to branch master-staging at 2:37:18 PM on Apr 8, 2026
Started 57 min ago
Took 1 min 12 sec
This run spent:
8.7 sec waiting;
1 min 12 sec build duration;
1 min 20 sec total from scheduled to completion.
Revision: adc6db5c405d8249c18cd88f33f7a0212c70bd8e
Repository: https://github.com/VIINIKKA/secure-programming-llm.git
master-staging
Simplify post-setup 2FA UI
REST API
Jenkins 2.541.2

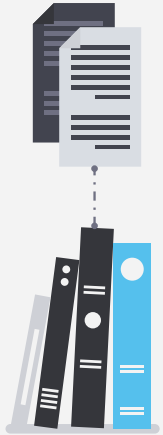


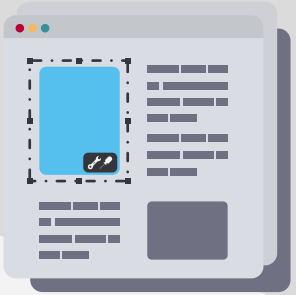
11

AI Usage

AI Usage

- AI assisted with coding support, documentation improvements, and testing ideas.
- Final design, implementation review, testing, and integration were done manually.
    - AI was used as a tool to learn more from the security domain





12

Continue on GitHub

Continue on GitHub

- https://github.com/VIINIKKA/secure-programming-llm
- Open these files first
- README.md and SECURITY.md